Machine learning: Decision tree (Cây quyết định)

Cây quyết định hồi quy

- Sử dụng hồi quy tuyến tính để dự báo giá nhà trên bộ dữ liệu được mô tả sau:
  The California housing dataset — Scikit-learn course

  Các công việc cần thực hiện:

  1. Báo cáo tổng quan dữ liệu
  2. Làm chuẩn dữ liệu (nếu có)
  3. Chia train/test (80/20, random_state=42).
  4. Sử dụng huấn luyện bằng LinearRegression.
  5. Báo cáo MAE, RMSE, R^2, r.
  6. Nhận xét kết quả và mô hình.

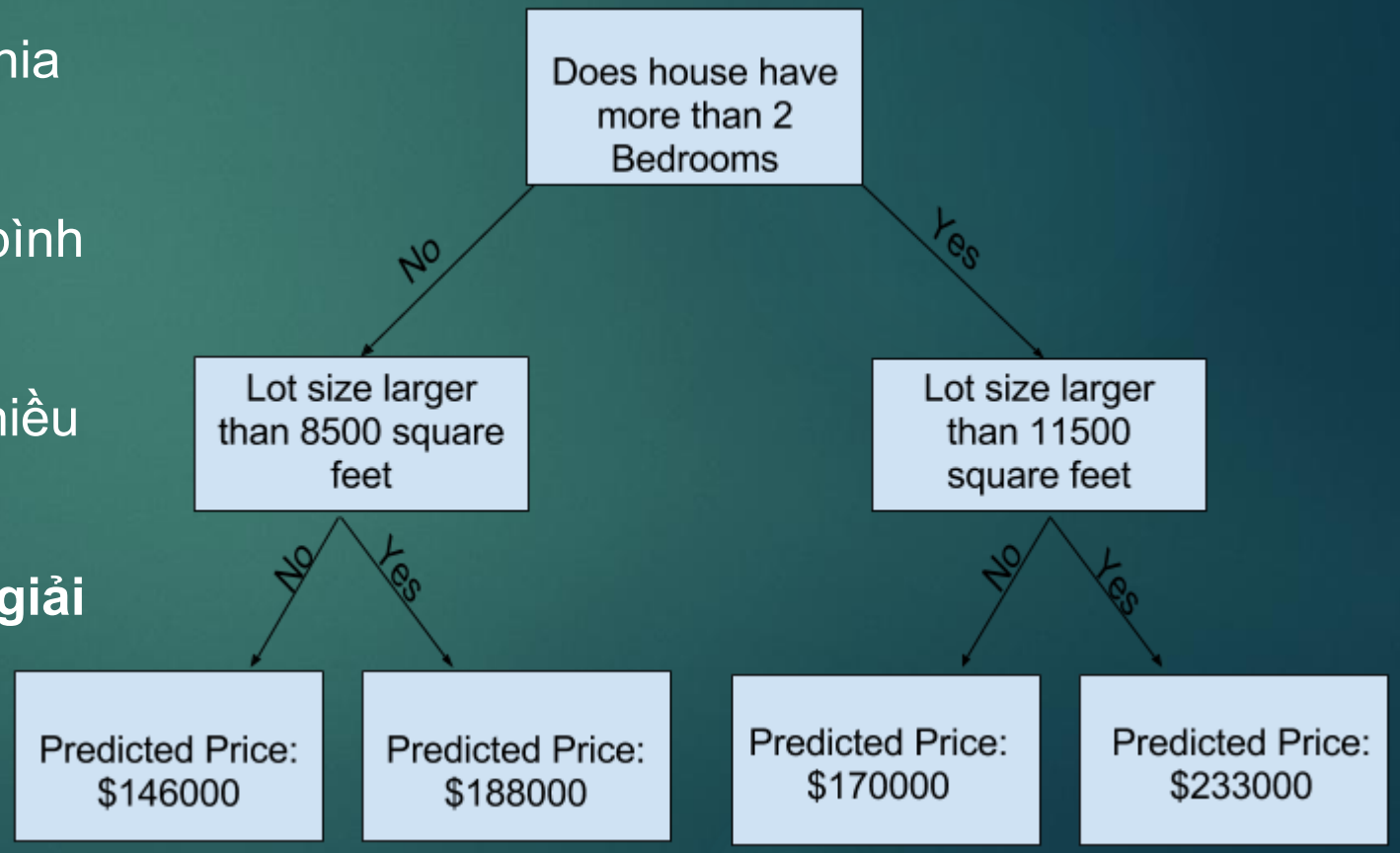
How model work: Introduction

Vấn đề: Dự đoán giá nhà: cây quyết định

Ý tưởng: lặp lại 2 bước

1. Chọn **đặc trưng** x_j và **ngưỡng** t để chia dữ liệu thành hai nhánh: $x_j \leq t$ và $x_j > t$.
2. Gán **giá trị dự báo** ở mỗi lá là trung bình $\bar{y}$ của các điểm trong lá.

- Đầu ra là **hàm bậc thang** theo từng chiều đặc trưng.
- Ưu điểm: mô hình **phi tuyến**, dễ **diễn giải** (đường đi quyết định).



How model work: Introduction

Tiêu chí tách nút

- Mục tiêu chọn split (j, t) để **giảm sai số** tối đa.
- Tiêu chí phổ biến (scikit-learn):
  - `squared_error` (MSE/variance decrease) — mặc định, mượt cho gradient-based pruning.
  - `absolute_error` (MAE) — bền hơn với ngoại lai.
  - `friedman_mse` — biến thể MSE giúp tối ưu boosting (thường dùng trong GB).
  - `poisson` — khi $y \geq 0$ và phân phối đếm.
- Tại mỗi lá L, dự báo $\hat{y}_L = \mathrm{mean}(y_i \in L)$.

How model work

Cách tìm split tối ưu

- Với một nút có tập S, thử đặc trưng x_j và ngưỡng t. Gọi:
  - $S_L = \{i \in S \mid x_{ij} \leq t\}$, $S_R = S \setminus S_L$.
- **Hàm mất mát MSE tổng**:

$$\mathrm{MSE}(S_L, S_R) = \sum_{i \in S_L} (y_i - \bar{y}_L)^2 + \sum_{i \in S_R} (y_i - \bar{y}_R)^2$$

- Chọn (j, t) làm **argmin** của MSE (hoặc argmax của "giảm phương sai").
- Thực tế: thử các ngưỡng ứng viên từ các giá trị duy nhất/điểm giữa của x_j.

How model work

Thuật toán học cây

- **Đệ quy từ gốc đến lá**:
  1. Nếu thoả điều kiện dừng → **tạo lá** với dự báo $\bar{y}$.
  2. Ngược lại: duyệt các (j, t), chọn split tốt nhất theo criterion.
  3. Gọi đệ quy cho hai nhánh trái/phải.
- **Điều kiện dừng** (pre-pruning): giới hạn độ sâu, số mẫu tối thiểu ở lá/để tách, độ giảm sai số tối thiểu, v.v.

How model work

Tham số quan trọng

- `max_depth` : độ sâu tối đa (khống chế độ phức tạp).
- `min_samples_leaf` : tối thiểu số mẫu ở một lá (tăng → làm mịn dự báo, giảm overfit).
- `min_samples_split` : tối thiểu số mẫu để xét tách một nút.
- `max_features` : số đặc trưng được xét khi tách (giảm → nhanh hơn, bền hơn với nhiễu).
- `min_impurity_decrease` : ngưỡng giảm sai số tối thiểu để chấp nhận split.

How model work

Đánh giá mô hình

- Chỉ số:
  - **RMSE** (đơn vị tiền tệ) — dễ diễn giải.
  - **MAE** — bền với ngoại lai.
  - R^2 — tỉ lệ phương sai giải thích.
- **K-Fold Cross-Validation** (K=5/10) để ước lượng ổn định.
- So sánh với **baseline tuyến tính/đa thức** để thấy lợi thế phi tuyến của cây.

Machine learning: Random Forest (Rừng ngẫu nhiên)

Vì sao cần rừng ngẫu nhiên

- **Cây quyết định** nếu để sâu, cây dễ học cả **nhiễu** ⇒ điểm huấn luyện rất cao, nhưng dự báo dữ liệu mới lại kém (**overfit**).
- **Cây dễ "nhạy cảm"**: chỉ cần đổi 1–2 mẫu dữ liệu, cấu trúc cây có thể thay đổi rất nhiều ⇒ kết quả không ổn định.
- **Rừng ngẫu nhiên = nhiều cây cộng lại**: ta huấn luyện nhiều cây *khác nhau*, rồi **lấy trung bình dự báo** ⇒ kết quả **ổn định** hơn, ít overfit hơn.
- **Thêm ngẫu nhiên để bớt tương tự**: mỗi cây học trên **một mẫu ngẫu nhiên** của dữ liệu (bootstrap) **và** chỉ nhìn **một số đặc trưng** khi tách ⇒ các cây đa dạng hơn, trung bình lên sẽ "mượt".
- **Dùng được ngay, ít phải tinh chỉnh**: Random Forest thường cho **kết quả tốt**, dễ dùng cho bài toán như dự báo giá nhà.

Random Forest là gì?

- Dùng **nhiều cây quyết định** cùng giải bài toán.
- Mỗi cây tự đưa ra một con số dự báo.
- **Kết quả cuối cùng = trung bình** của tất cả cây ⇒ bớt thiên lệch về một cây nào đó

- **Bootstrap theo mẫu**: mỗi cây học trên **bản sao xáo trộn** của tập huấn luyện (lấy mẫu có lặp).
- **Ngẫu nhiên đặc trưng khi tách** (`max_features`): khi chọn cách chia ở mỗi nút, cây **chỉ xét một số cột** chứ không xét hết ⇒ các cây nhìn vấn đề theo những "góc" khác nhau.

Quy trình học của Random Forest

- Lặp lại nhiều lần để tạo ra **nhiều cây**:
  - Rút mẫu bootstrap,
  - Huấn luyện cây (thường để khá sâu),
  - Tách nút chỉ xét **một phần đặc trưng**.
- Lúc dự báo: **trung bình** dự báo của mọi cây.

Các nút chỉnh quan trọng (siêu tham số)

- n_estimators (số cây): nhiều hơn ⇒ ổn định hơn (thường 300–1000 là đủ).
- max_depth: độ sâu tối đa; để None cho cây sâu, nhưng nếu thấy dấu hiệu overfit thì **giới hạn**.
- min_samples_leaf: số mẫu ít nhất ở một lá; tăng số này giúp **đỡ overfit**.
- min_samples_split: số mẫu tối thiểu để tách một nút (giúp bớt tách vụn).
- max_features: số cột được phép xét khi tách; **nhỏ vừa phải** giúp cây đa dạng hơn.
- bootstrap: nên để True (chuẩn bagging).

max_features ảnh hưởng thế nào

- **Nhỏ đi** → mỗi split chỉ nhìn ít cột → **các cây khác nhau hơn** → kết quả trung bình **tổng quát hóa tốt** hơn.
- **Quá nhỏ** → có thể **bỏ lỡ** cột mạnh ⇒ dự báo kém đi.

Đánh giá

- **RMSE** (đơn vị tiền tệ) — dễ hiểu vì cùng đơn vị với giá.
- **MAE** — bền với ngoại lai.
- **R^2** — phần trăm biến thiên giải thích được.
- Quy trình: chạy **K-Fold CV (K=5)** để có con số ổn định, **so sánh** Random Forest với **Cây đơn** để thấy lợi ích “bình quân nhiều cây”.

Minh họa cây quyết định, rừng ngẫu nhiên để dự báo giá nhà

Link dữ liệu: Housing Prices Competition for Kaggle Learn Users | Kaggle

Bước 1. Chuẩn bị, đọc dữ liệu

1. Tải dữ liệu từ Kaggle: cuộc thi Home Data for ML Course: https://www.kaggle.com/competitions/home-data-for-ml-course/data và đặt `train.csv`, `test.csv` vào thư mục `./home-data/`.
2. Đọc `train.csv`, kiểm tra kích thước, liệt kê một vài cột.
3. Xác định cột mục tiêu `SalePrice` và tách `X`, `y`.

Bước 2. Xây dựng pipeline tiền xử lý

- Với biến số: điền khuyết bằng median.
- Với biến phân loại: điền khuyết mode và One-Hot Encoding (bỏ qua nhãn lạ).
- Kết hợp bằng `ColumnTransformer`.

Bước 3. Baseline: Decision Tree với k-fold CV

- Xây dựng Pipeline(preprocess → DecisionTreeRegressor).
- Đánh giá bằng 5-fold CV với các chỉ số RMSE, MAE, R^2.

Bước 4. Baseline: **Baseline: Random Forest với K-fold CV**

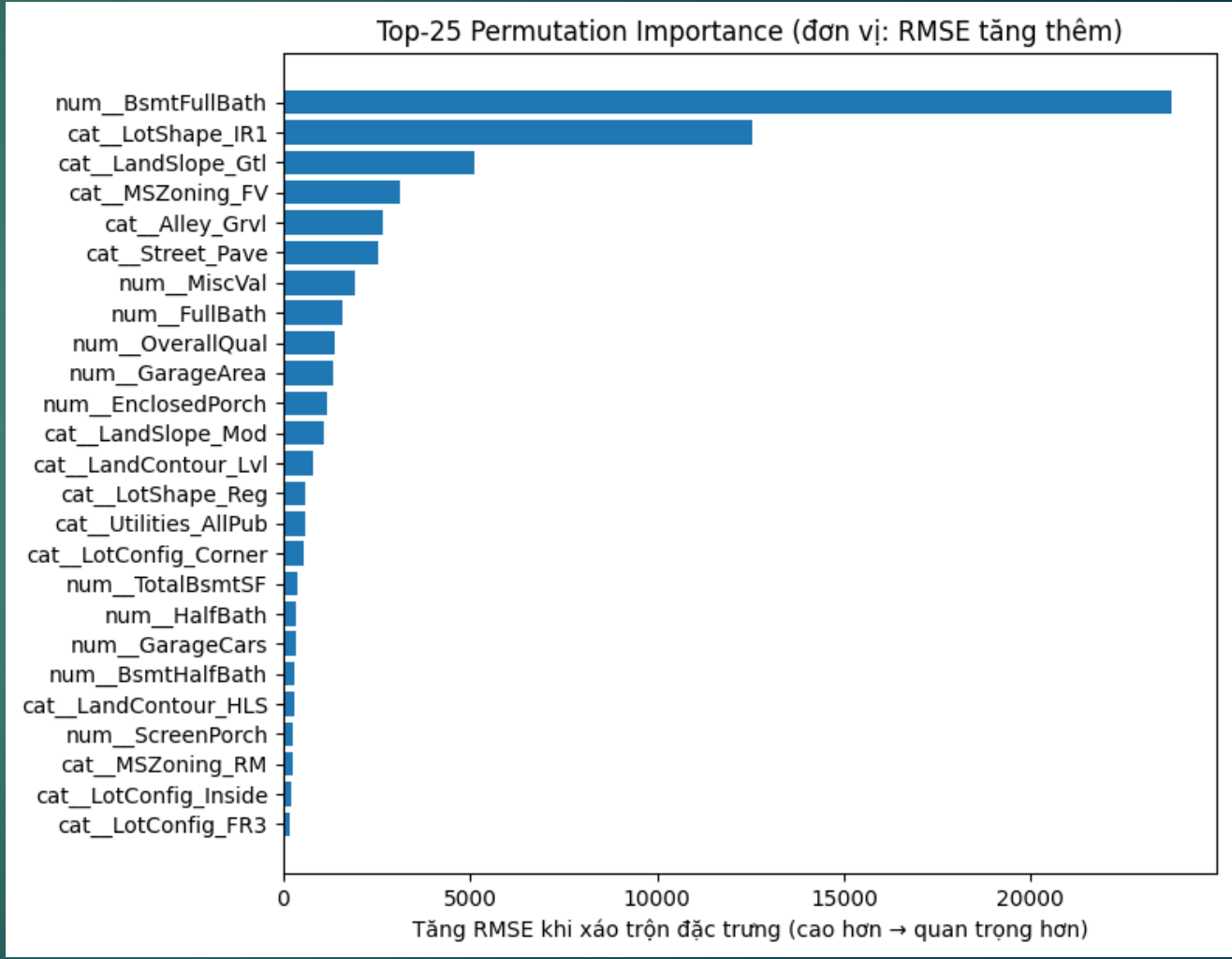
- Xây dựng Pipeline(preprocess → RandomForestRegressor(n_estimators=500)).
- So sánh điểm số với Decision Tree.

Bước 5. Tinh chỉnh siêu tham số

- Decision Tree: dùng GridSearchCV trên max_depth, min_samples_split, min_samples_leaf, max_features.
- Random Forest: dùng RandomizedSearchCV (ít nhất 20 tổ hợp) trên `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `bootstrap`.
- - Chọn mô hình có **CV RMSE** tốt nhất.

Bước 6. **Đánh giá hold-out & Permutation Importance**

- **Decision Tree**: dùng `GridSearchCV` trên `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`.
- **Random Forest**: dùng `RandomizedSearchCV` (ít nhất 20 tổ hợp) trên `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `bootstrap`. - Chia `train/validation` (ví dụ 80/20).
- Huấn luyện mô hình tốt nhất, báo cáo **RMSE/MAE/R²** trên validation.
- Tính **Permutation Importance**, in **Top-25** đặc trưng quan trọng và **vẽ biểu đồ cột ngang**.
- Chọn mô hình có **CV RMSE** tốt nhất.



Bước 6. **Huấn luyện full & tạo `submission.csv`**

- Huấn luyện mô hình tốt nhất trên **toàn bộ train**.
- Dự đoán `SalePrice` cho `test.csv` và ghi `./home-data/submission.csv`.

Bài tập

- Dự đoán churn khách hàng viễn thông bằng cây quyết định/rừng ngẫu nhiên
- link dữ liệu: Advanced Customer Churn Prediction